



数据结构

Data Structure

许 蕾

xlei@nju.edu.cn



请给出一个函数，计算 $1+2+\dots+n$ 的值

循环/迭代

```
#include <iostream>
using namespace std;

int sumUsingForLoop(int n) {
    int sum = 0; // 初始化求和变量
    for (int i = 1; i <= n; ++i) {
        sum += i; // 将每个数累加到 sum 中
    }
    return sum;
}

int main() {
    int n;
    cout << "请输入一个正整数 n: ";
    cin >> n;

    if (n < 1) {
        cout << "请输入大于0的正整数! " << endl;
        return 1;
    }

    int result = sumUsingForLoop(n);
    cout << "1+2+...+" << n << " = " << result << endl;

    return 0;
}
```

递归

```
int sumUsingRecursion(int n) {
    // 1. 递归基 (终止条件) : 当 n == 1 时, 和为 1
    if (n == 1) {
        return 1;
    }
    // 2. 递归步骤: sum(n) = n + sum(n-1)
    else {
        return n + sumUsingRecursion(n - 1);
    }
}

int main() {
    int n;
    cout << "请输入一个正整数 n: ";
    cin >> n;

    if (n < 1) {
        cout << "请输入大于0的正整数! " << endl;
        return 1;
    }

    int result = sumUsingRecursion(n);
    cout << "1+2+...+" << n << " = " << result << endl;

    return 0;
}
```



§3.2 栈与递归



3.2.1 递归的概念

- 若一个对象部分地包含它自己，或用它自己给自己定义，则称这个对象是递归的；
- 若一个过程直接地或间接地调用自己，则称这个过程是递归的过程。
- 以下三种情况常常用到递归方法。
 - ◆ 定义是递归的
 - ◆ 数据结构是递归的
 - ◆ 问题的解法是递归的



1. 定义是递归的



例如，阶乘函数

$$n! = \begin{cases} 1, & \text{当 } n = 0 \text{ 时} \\ n * (n-1)!, & \text{当 } n \geq 1 \text{ 时} \end{cases}$$

求解阶乘函数的递归算法

```
long Factorial(long n) {  
    if (n == 0) return 1;  
    else return n * Factorial(n-1);  
}
```



求解阶乘 $n!$ 的过程

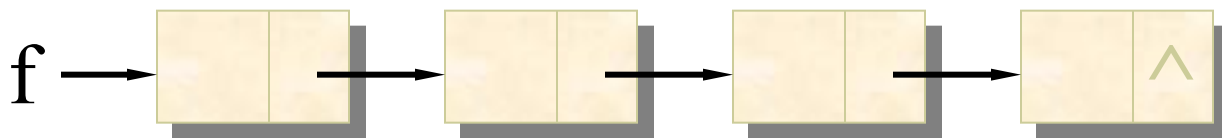
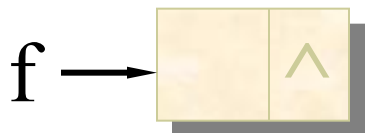




2. 数据结构是递归的



- 例如，单链表结构



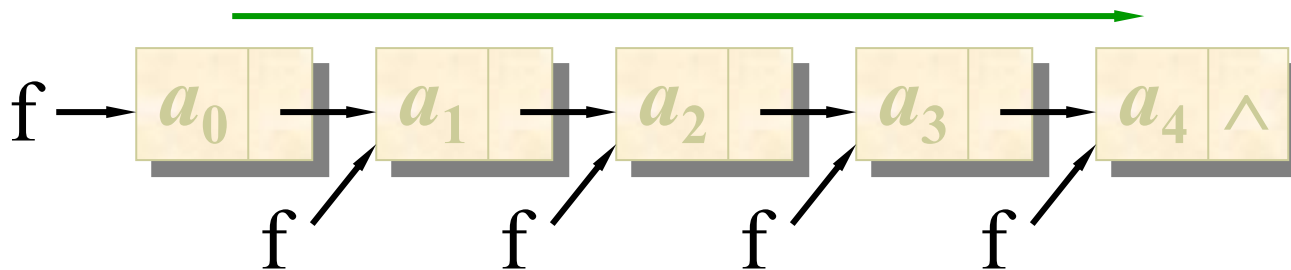
- 一个结点，它的指针域为NULL，是一个单链表；
- 一个结点，它的指针域指向单链表，仍是一个单链表。



搜索链表最后一个结点并打印其数值

```
template <class E>
void Print(ListNode<E> *f) {
    if (f -> link == NULL)
        cout << f -> data << endl;
    else Print(f -> link);
}
```

递归找链尾

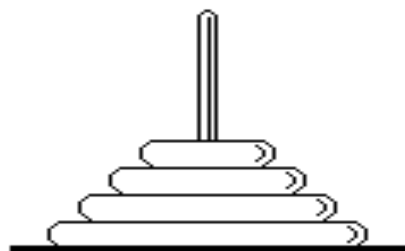




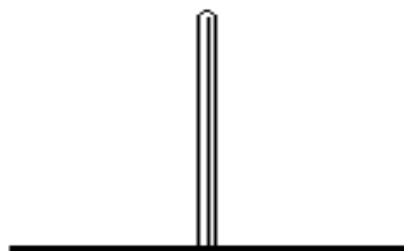
3. 问题的解法是递归的



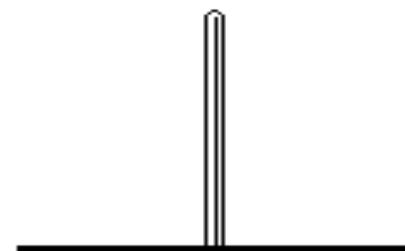
- 例，汉诺塔(Tower of Hanoi)问题的解法：
如果 $n = 1$ ，则将这一个盘子直接从 A 柱移到 C 柱上。否则，执行以下三步：
 - ① 用 C 柱做过渡，将 A 柱上的 $(n-1)$ 个盘子移到 B 柱上：
 - ② 将 A 柱上最后一个盘子直接移到 C 柱上；
 - ③ 用 A 柱做过渡，将 B 柱上的 $(n-1)$ 个盘子移到 C 柱上。



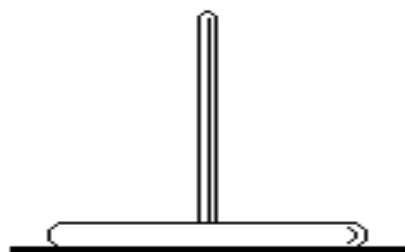
A



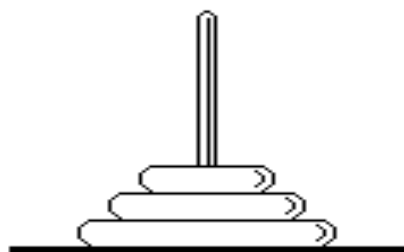
B



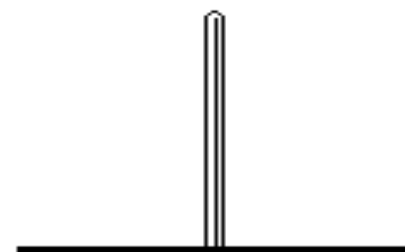
C



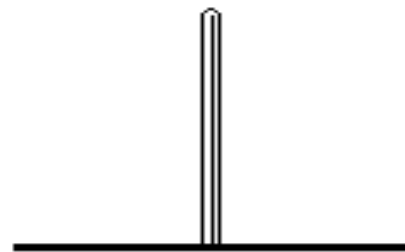
A



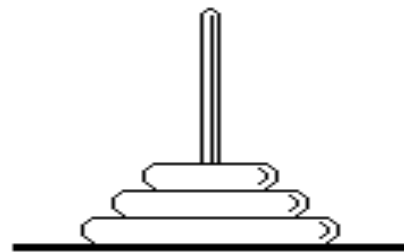
B



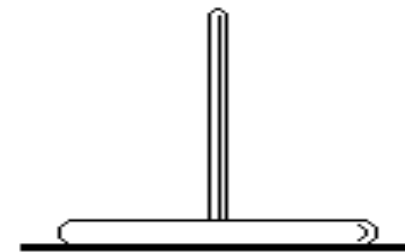
C



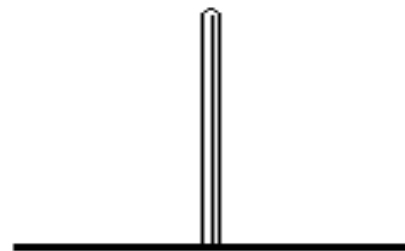
A



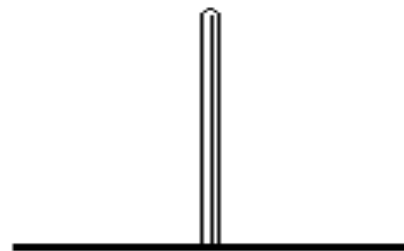
B



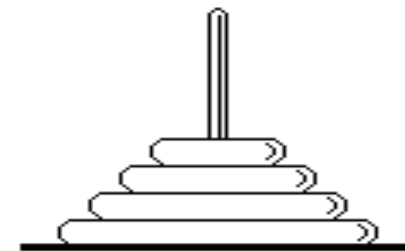
C



A



B



C



```
#include <iostream.h>
```

```
void Hanoi (int n, char A, char B, char C) {
```

```
//解决汉诺塔问题的算法
```

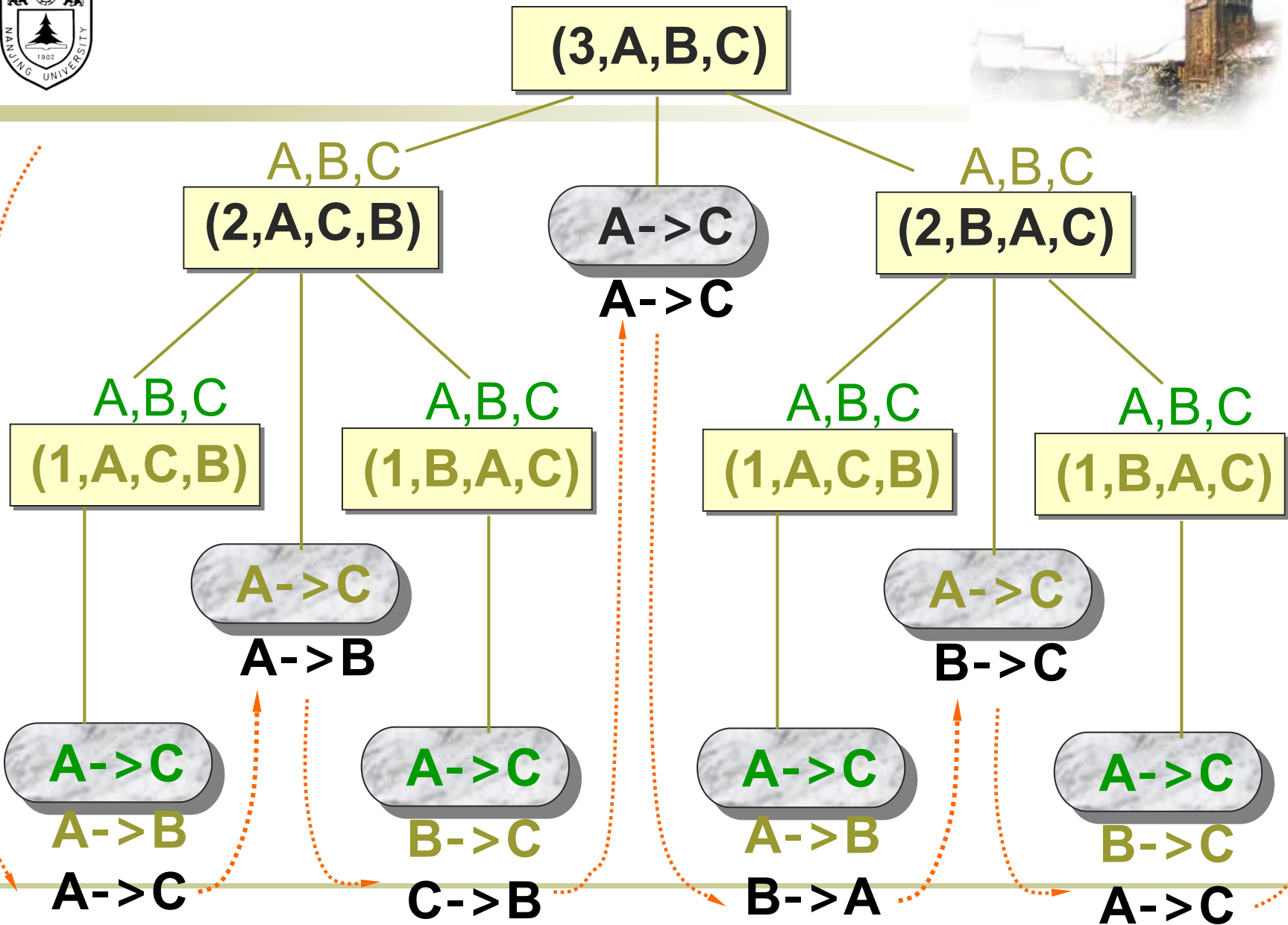
```
    if (n == 1) cout << " move " << A << " to " << C << endl;
```

```
    else { Hanoi(n-1, A, C, B); //以C柱为过渡  
          cout << " move " << A << " to " << C << endl;
```

```
          Hanoi(n-1, B, A, C); //以A柱为过渡
```

```
    }
```

```
}
```





什么时候运用递归？



- 子问题应与原问题做同样的事情，且更为简单；
- 把一个规模比较大的问题分解为一个或若干规模比较小的问题，分别对这些比较小的问题求解，再综合它们的结果，从而得到原问题的解。

— 分而治之策略（分治法）

- 这些比较小的问题的求解方法与原来问题的求解方法一样。



构成递归的条件



- 不能无限制地调用本身，必须有一个出口，化简为非递归状况直接处理。

```
Procedure <name> ( <parameter list> ) {  
    if ( < initial condition> ) //递归结束条件  
        return ( initial value );  
    else //递归  
        return ( <name> ( parameter exchange ));  
}
```



3.2.2 递归过程与递归工作栈



- 递归过程在实现时，需要自己调用自己。
- 层层向下递归，退出时的次序正好相反：

递归调用

$n! \longrightarrow (n-1)! \longrightarrow (n-2)! \longrightarrow \cdots \longrightarrow 1! \longrightarrow 0!=1$

返回次序

- 主程序第一次调用递归过程为**外部调用**；
- 递归过程每次递归调用自己为**内部调用**。
- 它们返回调用它的过程的地址不同。



```
long Factorial(long n) {  
    int temp;  
    if (n == 0) return 1;  
    else temp = n * Factorial(n-1);
```

```
RetLoc2 _____↑  
                return temp;  
            }
```

```
void main() {  
    int n;  
    n = Factorial(4);
```

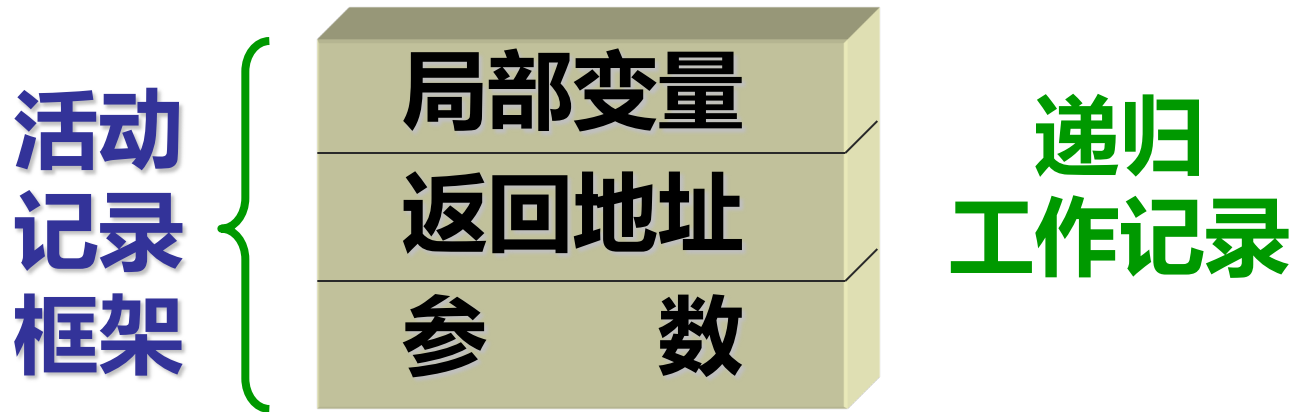
```
RetLoc1 _____↑  
            }
```



递归工作栈



- 每一次递归调用时，需要为过程中使用的参数、局部变量等另外分配存储空间。
- 每层递归调用需分配的空间形成递归工作记录，按**后进先出的栈**组织。



递归工作栈



调用块

.....
<下一条指令>

函数块

Function(<参数表>)
.....
<return>

函数递归时的活动记录

返回地址(下一条指令)	局部变量	参数
-------------	------	----



函数调用的背后（栈）



```
170 int func2 (int x) {  
171     int m, n;  
172     m = x + 1;  
173     n = x + 2;  
174 }  
175  
176 int func1 (int a, int b) { a: 1 b: 2  
177     int x= a+b; x: 3  
178     func2 (x);  
179     x = x+10086;  
180 }  
181  
182 int main() {  
183     int a = 1, b = 2, c = 3;  
184     func1(a, b);  
185     c = a+b;  
186 }
```

Debug: Stack

Debugger Console

Frames

Thread-1-<c...ain-thread>

func2(int) main.cpp:173

func1(int, int) main.cpp:178

main main.cpp:184

start 0x00007fff713d13d5

Variables LLDB

x = {int} 3

m = {int} 4

n = {int} 0

Frames

Thread-1-<c...ain-thread>

func2(int) main.cpp:173

func1(int, int) main.cpp:178

main main.cpp:184

start 0x00007fff713d13d5

Variables LLDB

a = {int} 1

b = {int} 2

x = {int} 3

Frames

Thread-1-<c...ain-thread>

func2(int) main.cpp:173

func1(int, int) main.cpp:178

main main.cpp:184

start 0x00007fff713d13d5

Variables LLDB

a = {int} 1

b = {int} 2

c = {int} 3



计算Factorial时活动记录的内容



递归调用序列

参数 返回值 返回地址

4	24	RetLoc1
---	----	---------

3	6	RetLoc2
---	---	---------

2	2	RetLoc2
---	---	---------

1	1	RetLoc2
---	---	---------

0	1	RetLoc2
---	---	---------

返回时的指令

return 4*6 //返回 24

return 3*2 //返回 6

return 2*1 //返回 2

return 1*1 //返回 1

return 1 //返回 1



递归过程改为非递归过程



- 递归过程简洁、易编、易懂
- 但递归过程效率低，重复计算多
- 改为非递归过程的目的是提高效率
- **单向递归**和**尾递归**可直接用迭代实现其非递归过程
- 其他情形必须借助**栈**实现非递归过程



计算斐波那契数列的函数Fib(n)的定义

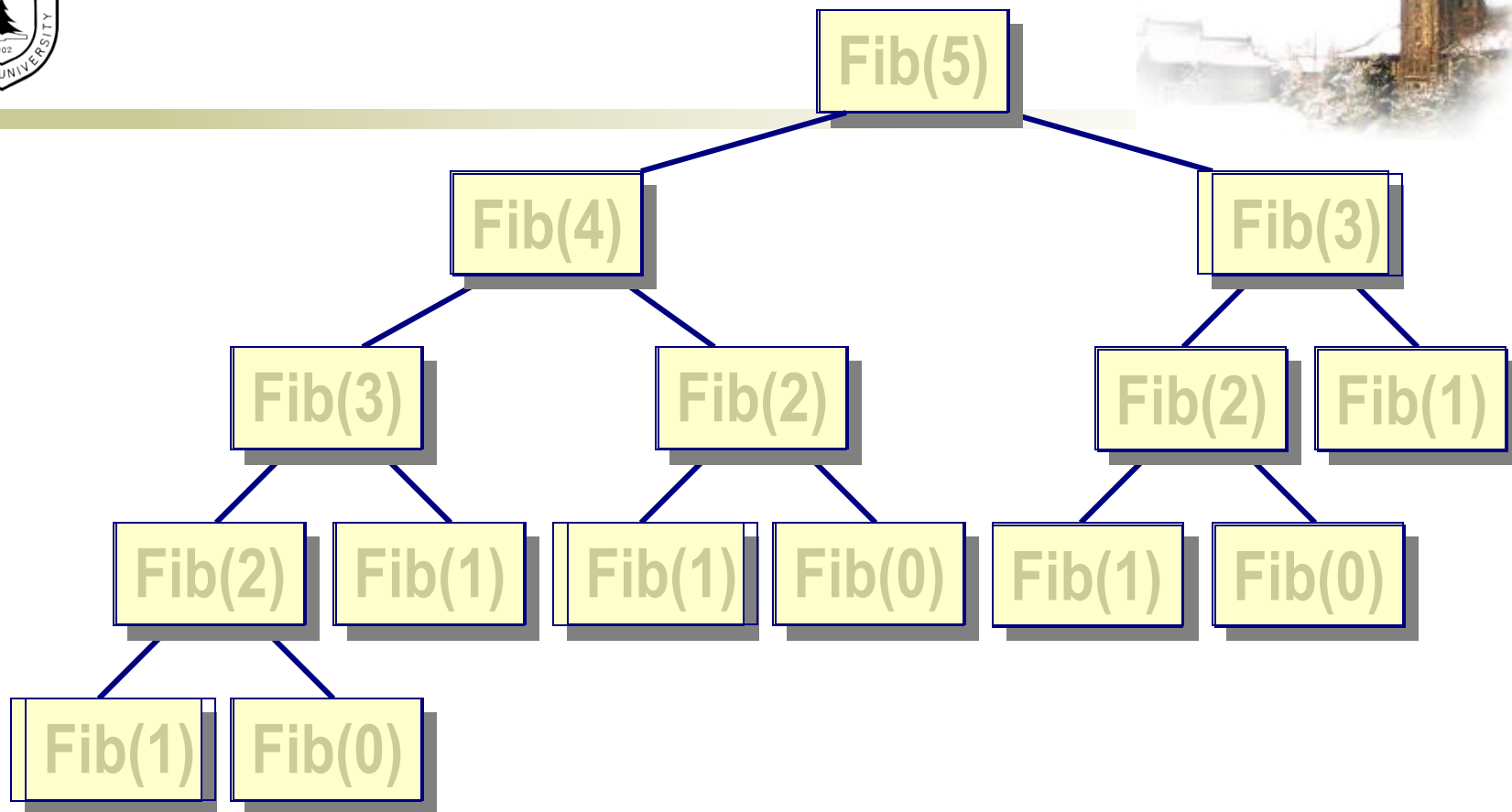


$$\text{Fib}(n) = \begin{cases} n, & n = 0, 1 \\ \text{Fib}(n-1) + \text{Fib}(n-2), & n > 1 \end{cases}$$

如 $F_0 = 0, F_1 = 1, F_2 = 1, F_3 = 2, F_4 = 3, F_5 = 5$

求解斐波那契数列的递归算法

```
long Fib(long n) {  
    if (n <= 1) return n;  
    else return Fib(n-1)+Fib(n-2);  
}
```



斐波那契数列的递归调用树

调用次数 $\text{NumCall}(k) = 2 * \text{Fib}(k+1) - 1$



单向递归用迭代法实现



```
long FibIter(long n) {  
    if (n <= 1) return n;  
    long twoback = 0, oneback = 1, Current;  
    for (int i = 2; i <= n; i++) {  
        Current = twoback + oneback;  
        twoback = oneback;  
        oneback = Current;  
    }  
    return Current;  
}
```



尾递归用迭代法实现（逆向打印数组）



25	36	72	18	99	49	54	63
----	----	----	----	----	----	----	----

```
void recfunc(int A[ ], int n) {  
    if (n >= 0) {  
        cout << A[n] << " ";  
        n--;  
        recfunc(A, n);  
    }  
}
```

先根据递归算法画出程序流程图
再建立起循环结构



```
void sterfunc(int A[ ], int n) {  
    //消除了尾递归的非递归函数  
    while (n >= 0) {  
        cout << "value    " << A[n] << endl;  
        n--;  
    }  
}
```